Reviewer Pack — Minimal Geometric Entropy of Metric Spaces and Communication…

Entry 0f0e855da629 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-04 23:21:20 UTC

Minimal Geometric Entropy of Metric Spaces and Communication Complexity Rank

Entry ID: `0f0e855da629` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-06-04 23:21:20 UTC

1. Conjecture

Field A (mathematical branch): Metric Geometry (Geometric Entropy) **Field B** (complexity object): Communication Complexity (Matrix Rank)

Statement:

> The geometric entropy of a metric space corresponding to an n-input Boolean function is lower-bounded by the rank of its communication complexity matrix, such that GEnt(Σ) ≥ k(n), where GEnt(Σ) is the geometric entropy of Σ and k(n) is a polynomial function of n.

Rationale (proposer's reasoning):

> Geometric entropy measures the information content of a metric space in terms of its curvature and dimension. Communication complexity captures the amount of communication needed to determine a property of a function. A higher geometric entropy indicates more complex structure, which could imply a higher rank for the communication matrix, reflecting the difficulty of communicating the function's properties.

Taxonomy category: `communication_complexity` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `84fca485304271ac`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> If for all tested Boolean functions with n ≤ 40, the geometric entropy GEnt(Σ) is greater than or equal to k(n), where k(n) is a polynomial function of n, then the conjecture is supported.

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.90	SAFE	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: `ADJACENT_OK` against 2 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - `"geometric entropy" AND "communication complexity rank"` - `"metric geometry" AND "Boolean function" AND communication complexity` - `"matrix rank" IN "geometric entropy" OF "metric spaces"`

Top relevant hits considered: - [s2:2512.21451] An approach to Fisher-Rao metric for infinite dimensional non-parametric information geometry - [s2:10.1101/2020.07.23.218289] Shape Dimensionality Metrics for Landmark Data

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_random_boolean_function(n):
        return [random.choice([0, 1]) for _ in range(2**n)]

    def nearest_neighbor_graph(boolean_function, n):
        graph = []
        for i in range(len(boolean_function)):
            distances = []
            for j in range(i + 1, len(boolean_function)):
                dist = sum(abs(a - b) for a, b in zip(bin(i)[2:].zfill(n), bin(j)[2:].zfill(n)))
                distances.append((j, dist))
            graph.append(distances)
        return graph

    def geometric_entropy(graph):
        n = len(graph)
        total_dist = 0
        count = 0
        for i in range(n):
            for j, dist in graph[i]:
                if dist == 1:
                    total_dist += 1
                    count += 1
        if count == 0:
```

```
        return 0
    avg_dist = total_dist / count
    entropy = -avg_dist * math.log2(avg_dist) - (1 - avg_dist) * math.log2(1 - avg_dist)
    return entropy

def communication_complexity_matrix(graph):
    n = len(graph)
    matrix = [[0] * n for _ in range(n)]
    for i in range(n):
        for j, dist in graph[i]:
            if dist == 1:
                matrix[i][j] = 1
                matrix[j][i] = 1
    return matrix

def rank(matrix):
    n = len(matrix)
    augmented_matrix = [row + [0] for row in matrix]
    for i in range(n):
        max_row = max(range(i, n), key=lambda x: abs(augmented_matrix[x][i]))
        augmented_matrix[i], augmented_matrix[max_row] = augmented_matrix[max_row], augmented_
        if augmented_matrix[i][i] == 0:
            continue
        for j in range(n + 1):
            augmented_matrix[i][j] /= augmented_matrix[i][i]
        for k in range(n):
            if k != i and augmented_matrix[k][i] != 0:
                factor = augmented_matrix[k][i]
                for j in range(n + 1):
                    augmented_matrix[k][j] -= factor * augmented_matrix[i][j]
    rank = sum(1 for row in augmented_matrix if any(row[j] != 0 for j in range(n)))
    return rank

n_values = [5, 10, 15, 20, 30, 40]
results = []

for n in n_values:
    boolean_function = generate_random_boolean_function(n)
    graph = nearest_neighbor_graph(boolean_function, n)
    g_ent = geometric_entropy(graph)
    comm_matrix = communication_complexity_matrix(graph)
    rnk = rank(comm_matrix)

    if rnk == 0:
        continue

    results.append({
        "n": n,
        "g_ent": g_ent,
        "rnk": rnk
    })

if not results:
    return {
        "metric_name": "geometric_entropy",
        "metric_value": None,
```

```
            "instances_tested": 0,
            "n_max": 0,
            "conjecture_holds": False,
            "counterexample": "no_valid_instances"
        }

    instances_tested = len(results)
    n_max = max(result["n"] for result in results)
    g_ent_values = [result["g_ent"] for result in results]
    rnk_values = [result["rnk"] for result in results]

    mean_g_ent = sum(g_ent_values) / instances_tested
    std_g_ent = math.sqrt(sum((x - mean_g_ent)**2 for x in g_ent_values) / instances_tested)

    conjecture_holds = all(g >= r for g, r in zip(g_ent_values, rnk_values))
    counterexample = "" if conjecture_holds else "g < r"

    return {
        "metric_name": "geometric_entropy",
        "metric_value": mean_g_ent,
        "instances_tested": instances_tested,
        "n_max": n_max,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] if sys.argv[1:] else [2, 3, 5, 7, 11, 13, 17, 19, 23, 2

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {{\"seed\": {seed}, ...{result}...}}")
        results.append(result)

    mean_g_ent = sum(r["metric_value"] for r in results if r["metric_value"] is not None) / len(re
    std_g_ent = math.sqrt(sum((r["metric_value"] - mean_g_ent)**2 for r in results if r["metric_va

    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_g_ent} std={std_g_ent} support_fraction={support_frac
    elif any(not r["conjecture_holds"] for r in results):
        counterexample = next(r["counterexample"] for r in results if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"{counterexample}\" first_failing_seed={seeds[ne
    else:
        print("RESULT: INCONCLUSIVE insufficient_data")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_30573715.py", line 132, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_30573715.py", line 82, in run_trial
    graph = nearest_neighbor_graph(boolean_function, n)
            ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_30573715.py", line 29, in nearest_neigh
    dist = sum(abs(a - b) for a, b in zip(bin(i)[2:].zfill(n), bin(j)[2:].zfill(n)))
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_30573715.py", line 29, in <genexpr>
    dist = sum(abs(a - b) for a, b in zip(bin(i)[2:].zfill(n), bin(j)[2:].zfill(n)))
                   ~~^~~
TypeError: unsupported operand type(s) for -: 'str' and 'str'
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> The test code crashed during execution, which prevents us from verifying the conjecture's support condition. | next: Investigate and fix the error in the test code to allow for a proper verification of the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	14114
2	propose	ollama_remote	glm4:latest	0	0	16359
3	preregistration	ollama_remote	glm4:latest	0	0	9091
4	novelty	ollama_remote	glm4:latest	0	0	8404
5	novelty	ollama_remote	glm4:latest	0	0	9462
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16515
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13924
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12831
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16248
10	judge	ollama_remote	glm4:latest	0	0	17164

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 134113 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in `research/audit/0f0e855da629.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/0f0e855da629.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/0f0e855da629.tar.gz` (if generated)